CortexOS Consolidated Spec (Draft v0.1)

*Consolidated from the two planning documents. Status: draft for review. Items marked **[Decided]** came from your planning; items marked **[Proposed]** are audit recommendations awaiting your approval. Part 2 of the planning document was cut off at the "Design Studio" section, so the agent operating-environment section is provisional.*

1. Vision

CortexOS is an AI-native operating system for building software. It coordinates a small team of specialist agents, stores everything they learn and produce in a structured second brain, and gives you a single interface to direct the work.

- **[Decided]** The War Room is the primary product (execution). The 3D Brain is the memory visualisation behind it.
- **[Decided]** The existing 4C foundation (Context, Connections, Capabilities, Cadence) stays as the kernel. It is not replaced.
- **[Proposed]** Apex alignment: decide whether the CortexOS Apex blueprint (context packs, architecture contract, verification gate, SQLite store) is this system's kernel, with the App Factory as a later layer. This must be settled before build starts.

2. Canonical names

The planning documents used several names for the same thing. This spec uses one each.

Canonical name	Replaces	Role
Jarvis (Supervisor)	Agent Router, Cortex Supervisor, Orchestrator	Plans, routes, sequences, enforces gates, resolves conflicts
Chief Librarian	Knowledge Steward, Knowledge & Memory Agent	Sole gate into the brain; maintains wiki, graph, index
Phase agents (7)	Phase Agents, Specialist Agents	Discovery, Product, Design, Architecture, Development, QA, Release
Second Brain	Brain, Knowledge Vault	The whole memory layer: raw, artifacts, wiki, graph

Canonical name	Replaces	Role
War Room	Mission Control, Agent Council	The execution interface
Artifact	Output, Asset	A typed, versioned output produced by an agent

Agent count: 9 roles total (Jarvis, Chief Librarian, 7 phase agents). Growth and analytics fold into Release. No micro-agents.

3. Architecture

```
YOU
  |
JARVIS (Supervisor)
  |-- Discovery | Product | Design | Architecture | Development | QA |
Release
  |
CHIEF LIBRARIAN  (nothing enters the brain except through here)
  |
SECOND BRAIN: Raw -> Artifacts -> Wiki -> Graph
```

Kernel (4C): Context, Connections, Capabilities, Cadence. **Memory layer:** Second Brain, Knowledge Graph, LLM Wiki, Chief Librarian. **Execution layer:** Jarvis and the 7 phase agents. **Interface layer:** Brain Mode and War Room.

Decisions on the contradictions in the source documents

- **[Proposed] Who talks to you?** Jarvis is the only agent that addresses you by default. The War Room council view shows agent contributions, but each one is posted *through* Jarvis, with attribution. You can open a direct line to any agent on request.
- **[Proposed]** The Second Brain is storage and does no work. The Chief Librarian maintains it.

4. Knowledge architecture

Three kinds of knowledge, stored differently. **[Decided]** The wiki is a derived layer and is never the source of truth.

Type	Examples	Storage	Mutability
Raw evidence	Meetings, transcripts, PDFs, repos, research	Files in `raw/`	Immutable
Operational memory	Project status, tasks, decisions, runs, agent state	**[Proposed]** Database	Changes constantly
Learned knowledge	Patterns, doctrines, playbooks, lessons	Markdown in `wiki/`	Maintained, with provenance

Source hierarchy (Decided): Raw sources, then project artifacts, then wiki knowledge, then graph intelligence.

Folder layout (Decided):

```
brain/
  raw/  inbox/  projects/  artifacts/
  wiki/ (entities, concepts, decisions, doctrines, patterns, playbooks,
summaries)
  graph/  agents/  memory/  logs/
```

Rules (Decided): project data lives in `projects/` and `artifacts/`; the wiki references it and holds lessons, doctrine and decisions about it. Keep names flat and rely on links and an index rather than deep nesting.

Proposed additions:

- **Operational data in a database** (tasks, runs, agent state, artifact metadata), because the source material itself warns that wikis break on structured multi-writer data.
- **The graph is a rebuildable index** generated from files and the database, never edited directly.
- **Every wiki claim carries provenance:** a link to the raw source or artifact, plus a review flag. Doctrine pages require your approval.

5. Chief Librarian

Built first among the agents. **[Decided]** All knowledge passes through it.

Responsibilities: ingest, classify, tag, extract entities and concepts, cross-link, summarise, detect contradictions, version, update wiki and graph, refresh index and search, audit.

Pipeline: New source or artifact, then save original, create metadata, extract entities and concepts, map relationships, update wiki, update graph, update index, audit.

Two core commands (from the LLM Wiki pattern): *Ingest* (file the source and ripple it through every relevant page) and *Query* (read the index first, open only relevant pages, answer with citations, separating what the wiki says from what the model adds).

Boundaries (adopted from the source rulebook): never modify raw files; never delete a page without asking (deprecate and link forward); never invent facts.

[Proposed] Safeguards:

- Ingestion is queued and idempotent, so one failure does not block the system.
- Ingested content is **data, never instructions** (prompt-injection defence for web and PDF sources).
- Health-check job for contradictions, stale claims, orphan pages and missing cross-references.

6. Jarvis (Supervisor)

Role (Decided): orchestrator, governor, project manager, quality manager. Agents never self-organise.

Responsibilities: routing, task assignment, sequencing, progress tracking, constraint enforcement, quality gates, conflict resolution, priority management, artifact approval, memory checks.

[Proposed] Design: a deterministic workflow engine (state machine, gates, retries, timeouts) with an LLM planner on top. The workflow is not pure LLM judgement.

[Proposed] Workflow: default order is Discovery, Product, Design, Architecture, Development, QA, Release, with **feedback edges** (QA to Development, Architecture to Product, Release to Discovery after launch). Independent steps can run in parallel (for example, Design prepares a UX framework while Discovery researches).

External wiring [Decided in intent]: Jarvis can be wired to outside apps such as EA Studio or Hermes. **[Proposed]** Use MCP for tool access behind a thin adapter layer. This needs each app's real interface (API, CLI, or files) before the method can be chosen.

7. Phase agents

Agent	Handles	Produces
Discovery	Problem definition, validation, market and competitor research, user research	Validated opportunity, research report, personas
Product	Requirements, PRD, user stories, acceptance criteria, roadmap, MVP scope	PRD, backlog, roadmap, MVP spec
Design	UX, flows, wireframes, UI, design system	Flows, screens, components, design specs
Architecture	System, database, API, security and infrastructure design	Blueprint, ERD, API contracts, infrastructure diagram
Development	Frontend, backend, database, APIs, refactoring, documentation	Working code, repositories, services
QA	Unit, integration, regression, accessibility, security tests	Bug reports, coverage, test results
Release	CI/CD, deployment, monitoring, analytics, feedback, iteration	Releases, dashboards, metrics, improvement plans

Agent operating environment (provisional; source section was cut off)

[Decided] An agent is the smallest piece. Each phase gets a full environment, as in v0's design setup (design.md, skills, plugin, architecture, constraint rules, agent.md). Example: Design Agent becomes Design Studio.

[Proposed] Agent pack format, one folder per agent:

- `AGENT.md` : role, scope, inputs and outputs
- `constraints.md` : guardrails and what the agent must never do
- `skills/` : reusable procedures
- `tools/` : permitted tools and integrations
- `templates/` : artifact templates
- `evals/` : tests that define "working correctly"

8. Artifacts and handoffs

[Proposed] Agents hand off typed artifacts, not chat. Each artifact has a schema with required fields, an owning agent, a version, upstream sources, a status (draft, in review, approved) and an approval record.

Example: a PRD must contain problem, personas, MVP scope, user stories with acceptance criteria and non-goals before Jarvis lets Architecture start.

Graph node kinds (Decided): existing (skill, project, note, memory, routine, app, run) plus agent, artifact, decision, entity, concept, doctrine, pattern, project-phase.

9. Interface

Toggle (Decided): a small switch between Brain Mode and War Room, on an otherwise clean front screen.

Brain Mode: the 3D graph with the Second Brain at centre, agents and artifacts around it.

War Room layout (Decided):

- **Left, Research panel** (NotebookLM-style): web search and browser, add URL, PDF, docs, GitHub repo, notes, bookmarks, voice and meeting notes. Everything dropped here goes to the Chief Librarian first.
- **Centre, Mission Control:** project, plan, status, discussion, agent activity. The Jarvis summary stays pinned.
- **Right, Workbench:** apps, artifacts, tools. Detachable, resizable, dockable (VS Code style). Includes an agent activity feed.

[Proposed] Progress indicators in the activity feed must reflect real run states (queued, running, blocked, failed, done), not estimated percentages.

10. Governance and security

All **[Proposed]**; none of this was in the source documents.

- Least-privilege tools per agent; secrets never enter agent context.
- Human approval required before deploy and before any destructive action.
- Append-only audit log of agent actions and tool calls.
- Per-project isolation of knowledge where client data is involved.
- Cost and token budgets per run, with Jarvis stopping runaway loops.

- Versioned agent prompts and packs, with evals run on every change.
- Quality gates are executable (typecheck, lint, tests, schema validation), not LLM opinion.

11. Build plan

Source plan [Decided]: dependency order, Knowledge Foundation, then Librarian, then Jarvis, then agents, then War Room, over five months. Rule: never build the War Room, the Librarian or the agents first.

[Proposed] Revision: a vertical slice first.

1. **Foundation:** folder structure, artifact schemas, operational database, graph generation.
2. **Slice:** Chief Librarian plus Jarvis (terminal interface) plus Discovery, Product and Architecture on one real project, producing approved artifacts that land in the wiki.
3. **Expand:** Design, Development, QA, Release, with governance and evals.
4. **War Room UI** on top of a working backend.
5. **Polish:** detachable panels, workbench tools, audit engine.

The workbench tools (PRD, architecture, database, API and wireframe builders) are each substantial products. Treat them as a backlog, not a month-4 deliverable.

12. Open questions

1. **Target user:** only you, or a team and clients later? This drives isolation, auth and scale.
2. **Model and runtime:** the terminal currently calls Gemini. Will agents run on Gemini, Claude, or a mix? Which runtime executes agent loops?
3. **Apex relationship:** is Apex the kernel of this system, or a separate product?
4. **EA Studio and Hermes:** what interfaces do they expose?
5. **Jarvis authority:** which approvals can Jarvis grant itself, and which always need you?
6. **Operational store:** SQLite locally first, or a server database from the start?
7. **Definition of v1 done:** what single end-to-end result proves the system works?
8. **Missing source:** the rest of planning part 2, including the Design Studio and operating-environment section.

9. **Source code access:** the plan describes files such as `CortexTerminal.tsx` and `lib/knowledge-data.ts`. Share them so the node schema and routing hooks can be confirmed rather than assumed.

13. Deferred

More than seven phase agents, multi-user collaboration, marketplace for agent packs, voice interface, automated daily web digests.